

Library Management System

Analytics & ETL

Jeremy Roalef, Michael Roberts

CITA 345

May 12, 2025

Table of Contents

Table of Contents.....	1
Introduction	3
ERD	3
Relational Schema.....	4
Star Schema.....	5
Operational Data Dictionary	6
Analytical Data Dictionary	8
SQL Code for ETL	10
INSERT INTO Author.....	10
INSERT INTO Book	11
INSERT INTO Category	11
INSERT INTO Member	11
INSERT INTO Calendar	12
INSERT INTO BridgeBookCategoryFact.....	12
INSERT INTO BorrowFact.....	13
INSERT INTO ReservationFact.....	14
INSERT INTO ReviewFact.....	15
SQL Code for ETL Procedures	15
Create Procedure RefreshDimAuthor	16
Create Procedure RefreshDimBook	16
Create Procedure RefreshDimCategory.....	17
Create Procedure RefreshDimMember.....	17
Create Procedure RefreshFactBridgeBookCategory	18
Create Procedure RefreshFactBorrow	18
Create Procedure RefreshFactReservation	19
Create Procedure RefreshFactReview	20
Create Procedure RefreshWarehouse	21

SQL Code for Analytics	21
Find the total number of books borrowed/month-year	22
Find the average borrowing duration per genre	22
Find the % of late returned books per month	23
Find the average rating per genre	23
Identify Peak Borrowing Periods During the Week	24
SQL Code for Randomized Entries	24
INSERT Many INTO Author	24
INSERT Many INTO Book	26
INSERT Many INTO Category	27
INSERT Many INTO Member	28
INSERT Many INTO BookCategory	29
INSERT Many INTO Borrowing	30
INSERT Many INTO Reservation	30
INSERT Many INTO Review	31
Reflection	31

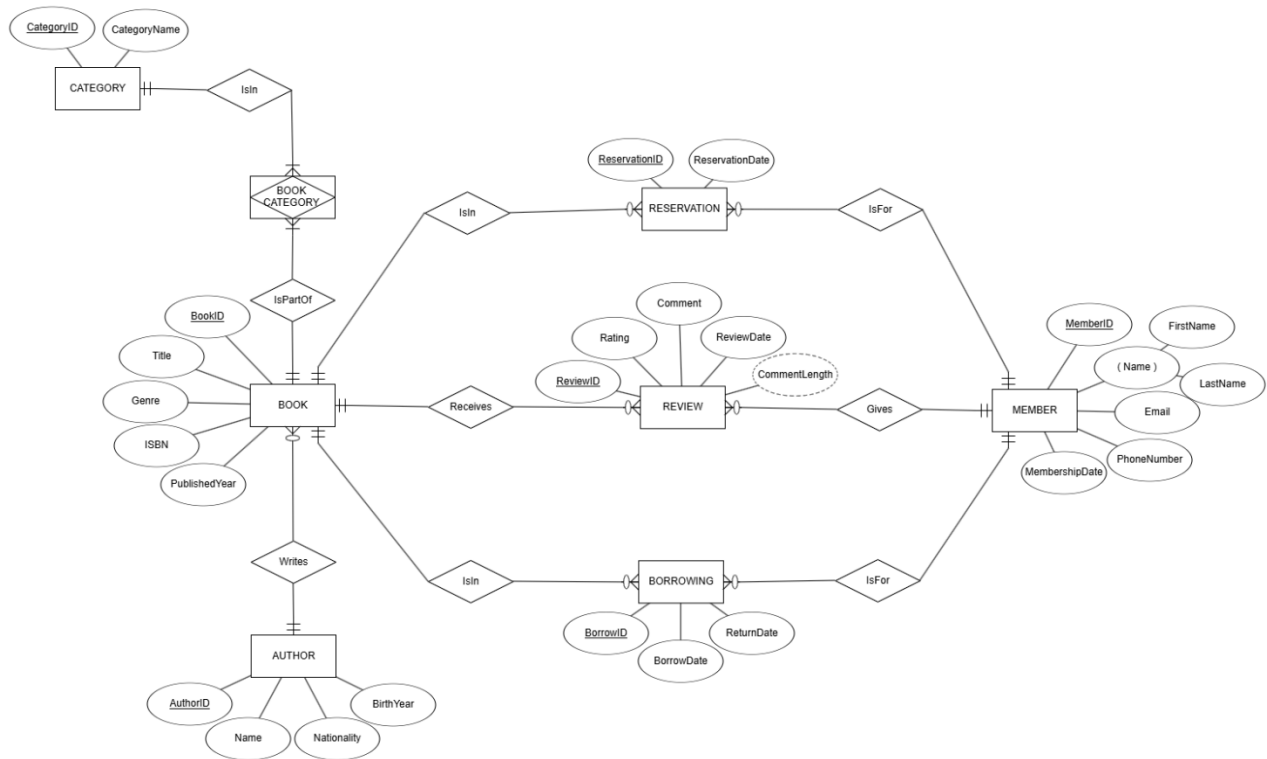
Introduction

The *Library Management System – Analytics & ETL* project explores the principles of data warehousing. We are given the queries for creating and populating tables in the operational database. From these queries, we must expand upon them and use extraction, transformation, and loading (ETL) to move the data into a warehouse for analytical purposes. Additionally, by designing a star schema, we can easily visualize the ETL process and reference it when writing analytical queries. Finally, the operational database will be populated with randomized data to further test the speed and quality of analytical querying.

Although not necessary for the project requirements, we will use AI tools to explore the usage of AI in databases. AI tools will not serve as a replacement for our work. Rather, we will use AI to enhance the quality of the project in an appropriate manner. For transparency, we will also specify when and where we used AI for our project, and we will explain what we've learned from using AI.

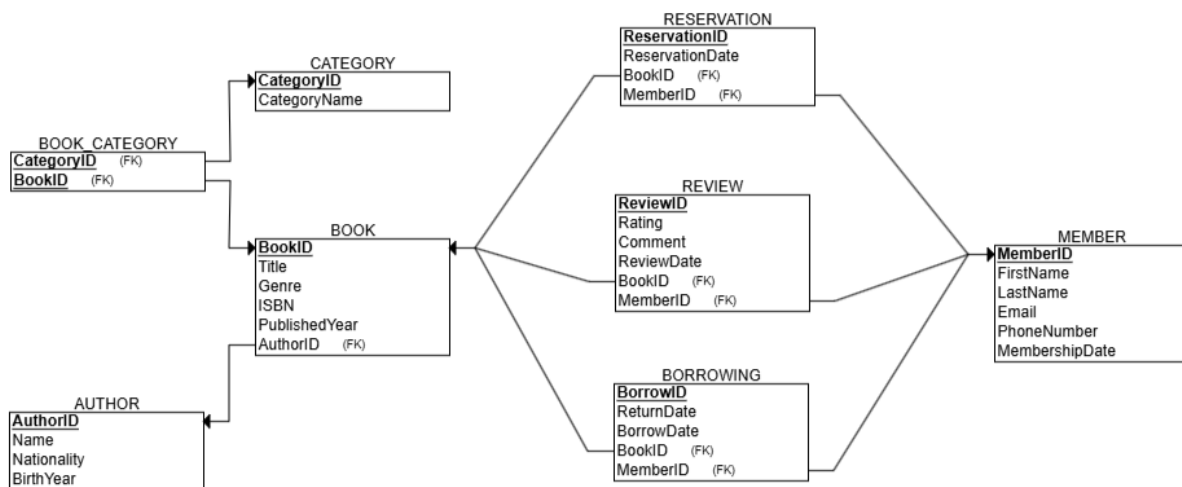
ERD

The entity-relationship diagram (ERD) below was created for visualization purposes. The ERD helps visualize the relationships between entities in the operational database that may be harder to notice in other visualization techniques. For that reason, we created what we believe to be the ERD from the given CREATE TABLE queries.



Relational Schema

The relational schema below depicts the tables in the operational database and their foreign key connections. Though not part of the requirements, the schema was useful in helping us determine how data relates to one another. This was especially helpful during the ETL process, when we must connect and extract tables through their foreign key connections before transforming the data to conform to warehousing standards.

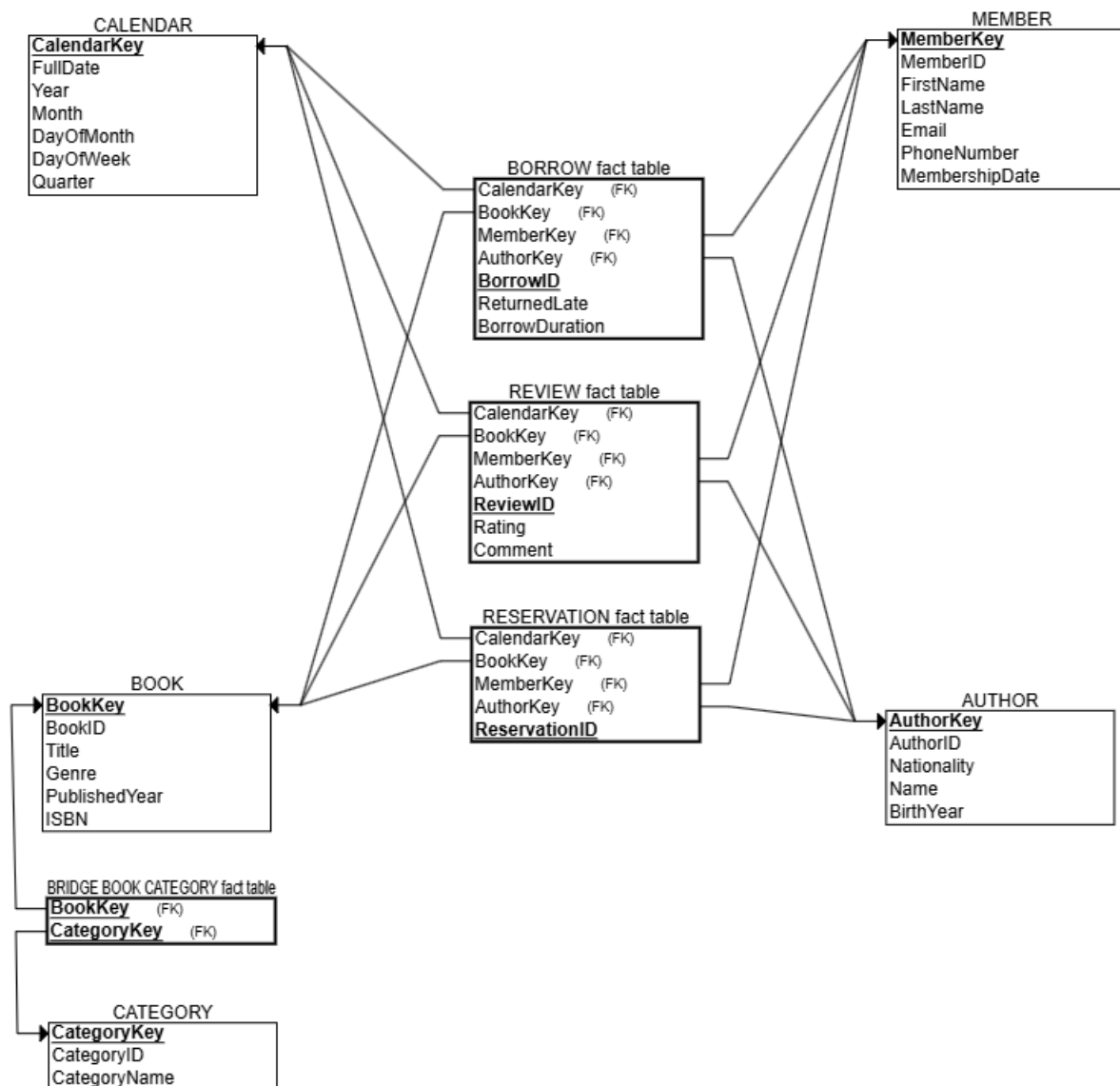


Star Schema

The star schema below depicts the relevant information in the operational database for analytical purposes. Each dimension's (calendar, member, author, and book) columns can be retrieved from the operational database. For example, the Member dimension is an exact replica of the Member table in the operational database, except for the member key in the dimension.

While designing the database, we noticed a problem with normalizing data with book and category. Generally, a book would have a category column in its table. However, since a book can have multiple categories, a separate table for categories would be generated, and an associative entity between Book and Category would solve normalization issues. However, this becomes an issue for the data warehouse, where we want to denormalize data because we are generally unconcerned about delete and update anomalies. If we denormalize the data, we would have many repeat entries in our book dimension. The solution to this is to use a "bridge" fact table, or otherwise called a "factless" fact table, which serves as a connecting link between two dimensions to prevent redundancy.

AI tools were used to brainstorm solutions to the denormalization problem explained above. Upon questioning AI tools like ChatGPT, we learned that our situation was a textbook case for using a bridge fact table. Our problem was caused by a gap in our general knowledge of star schema design.



Operational Data Dictionary

The operational data dictionary is another visualization design we implemented for ease of reference. Unlike the ERD, relational schema, or the star schema, a data dictionary provides sample rows in the database we can use to test our queries for correctness.

CATEGORY	
CategoryID	CategoryName
1	Fiction
2	Classic

AUTHOR			
AuthorID	Name	Nationality	BirthYear
1	J.K. Rowling	British	1965
2	George Orwell	British	1903
3	Jane Austen	British	1775

MEMBER					
MemberID	FirstNmae	LastName	Email	PhoneNumber	MembershipDate
1	Alice	Smith	alice@example.com	1234567890	1/10/2020
2	Bob	Johnson	bob@example.com	987654321	6/23/2019

BOOK					
BookID	Title	AuthorID	Genre	PublishedYear	ISBN
1	Harry Potter and the Philosopher's Stone	1	Fantasy	1997	9780747532699
2	1984	2	Dystopian	1949	9780451524935
3	Pride and Prejudice	3	Romance	1813	9780141439518

BORROWING					
BorrowID	BookID	MemberID	BorrowDate	ReturnDate	
1	1	1	1/10/2023	1/20/2023	
2	2	2	1/15/2023	1/25/2023	

RESERVATION			
ReservationID	BookID	MemberID	ReservationDate
1	3	1	2/1/2023

REVIEW					
ReviewID	BookID	MemberID	Rating	Comment	ReviewDate
1	1	1	5	Great book!	1/21/2023
2	2	2	4	Thought-provoking	1/26/2023

BOOK CATEGORY	
BookID	CategoryID
1	1
2	2
3	2

Analytical Data Dictionary

The analytical data dictionary includes what we expect to see in the analytical data warehouse after performing the ETL process on the operational database. We can use this data to confirm that our ETL queries are correct and ready for a mass scale of data transformation.

REVIEW Fact Table						
ReviewID	BookKey	MemberKey	AuthorKey	CalendarKey	Rating	Comment
1	1	1	1	3	5	Great book!
2	2	2	2	4	4	Thought-provoking

BRIDGE BOOK CATEGORY Fact Table	
BookKey	CategoryKey
1	1
2	2
3	2

CATEGORY Dimension		
CategoryKey	CategoryID	CategoryName
1	1	Fiction
2	2	Classic

AUTHOR Dimension				
AuthorKey	AuthorID	Name	Nationality	BirthYear
1	1	J.K. Rowling	British	1965
2	2	George Orwell	British	1903
3	3	Jane Austen	British	1775

RESERVATION Fact Table				
ReservationID	BookKey	MemberKey	AuthorKey	CalendarKey
1	3	1	3	5

CALENDAR Dimension						
CalendarKey	FullDate	Year	Month	DayOfMonth	DayOfWeek	Quarter
1	1/10/2023	2023	1	10	Tuesday	1
2	1/15/2023	2023	1	15	Sunday	1
3	1/21/2023	2023	1	21	Saturday	1
4	1/26/2023	2023	1	26	Thursday	1
5	2/1/2023	2023	2	1	Wednesday	1

BOOK Dimension					
BookKey	BookID	Title	Genre	PublishedYear	ISBN
1	1	Harry Potter and the Philosopher's Stone	Fantasy	1997	9780747532699
2	2	1984	Dystopian	1949	9780451524935
3	3	Pride and Prejudice	Romance	1813	9780141439518

BORROW Fact Table						
BorrowID	BookKey	MemberKey	AuthorKey	CalendarKey	BorrowDuration	ReturnedLate
1	1	1	1	1	10	0
2	2	2	2	2	10	0

SQL Code for ETL

The SQL code below is separated based on the tables that it's used for. Generally, we would use software and programming to aid in the ETL process, but we used SQL queries for this ETL process. The ETL process is more complicated than it may first seem. First, we must extract the data from the operational database, ensuring correct information is grabbed out of the operational database. Second, we must ensure that the data conforms to the analytical data warehouse standards. Finally, we must ensure that no duplicate entries are inserted into the warehouse by using the EXCEPT clause.

AI tools were used to brainstorm methods of referencing the operational database and the analytical data warehouse. We learned that there is a general syntax that we may follow when specifying where we want to insert data into tables and selecting data from tables, which significantly helped reference our operational and analytical database in the same query. Additionally, we consulted AI tools to brainstorm methods of excluding records in the operational database that have already been inserted in the analytical database, which is especially helpful for refresh loading into the warehouse. We learned that we can use the EXCEPT clause to determine whether a record exists within a table of records, much like we would use the NOT IN operator to determine whether a column value is in a table of values or not.

INERT INTO Author

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR (AuthorID, Name, Nationality, BirthYear)
SELECT opA.AuthorID, opA.name, opA.Nationality, opA.BirthYear
FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opA
EXCEPT
SELECT anaA.authorID, anaA.name, anaA.nationality, anaA.BirthYear
FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaA
```

INSERT INTO Book

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK
    SELECT opB.BookID, opb.Title, opb.Genre, opb.PublishedYear, opb.ISBN
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
    EXCEPT
    SELECT anaB.BookID, anaB.Title, anaB.Genre, anaB.PublishedYear, anaB.ISBN
    FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
```

INSERT INTO Category

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY
    --Row Selection
    SELECT opC.CategoryID, opC.CategoryName
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Category opC
    --Exclusion of existing rows
    EXCEPT
    SELECT anaC.CategoryID, anaC.CategoryName
    FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY anaC
```

INSERT INTO Member

```
--Insertion into Member dimension
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER
    --Row Selection
    SELECT opM.MemberID, opM.FirstName, opM.LastName, opM.Email, opM.PhoneNumber, opM.MembershipDate
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Member opM
    --Exclusion of existing data
    EXCEPT
    SELECT anaM.MemberID, anaM.FirstName, anaM.LastName, anaM.Email, anaM.PhoneNumber, anaM.MembershipDate
    FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
```

INSERT INTO Calendar

```
WITH DateRange AS (  
    SELECT CAST('2010-01-01' AS DATE) AS CalendarDate  
    UNION ALL  
    SELECT DATEADD(DAY, 1, CalendarDate)  
    FROM DateRange  
    WHERE CalendarDate < '2030-12-31'  
)  
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR  
SELECT  
    CalendarDate AS FullDate,  
    YEAR(CalendarDate) AS Year,  
    MONTH(CalendarDate) AS Month,  
    DAY(CalendarDate) AS DayOfMonth,  
    DATENAME(WEEKDAY, CalendarDate) AS DayOfWeek,  
    DATEPART(QUARTER, CalendarDate) AS Quarter  
FROM DateRange  
OPTION (MAXRECURSION 32767); -- Needed for large date ranges
```

INSERT INTO BridgeBookCategoryFact

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BRIDGEBOOKCATEGORYFACT  
--Row Selection  
SELECT anaBook.BookKey, anaCategory.CategoryKey  
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.BookCategory opBookCategory  
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaBook  
    ON anaBook.BookID = opBookCategory.BookID  
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY anaCategory  
    ON anaCategory.CategoryID = opBookCategory.CategoryID  
--Exclusion of existing rows  
EXCEPT  
SELECT anaBBCF.BookKey, anaBBCF.CategoryKey  
FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BRIDGEBOOKCATEGORYFACT anaBBCF
```

INSERT INTO BorrowFact

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BORROWFACT
--SELECTION
SELECT
    opBOR.BorrowID,
    anaB.BookKey,
    anaM.MemberKey,
    anaAU.AuthorKey,
    anaCAL.CalendarKey,
    DATEDIFF(DAY, opBOR.BorrowDate, opBOR.ReturnDate) AS BorrowDuration,
    CAST(CASE WHEN DATEDIFF(DAY, opBOR.BorrowDate, opBOR.ReturnDate) > 14 THEN 1 ELSE 0 END AS bit) AS ReturnedLate
FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Borrowing opBOR
--Grab reference to author
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
    ON opB.BookID = opBOR.BookID
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
    ON opAU.AuthorID = opB.AuthorID
--Retrieve key values from warehouse
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
    ON anaB.BookID = opB.BookID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
    ON anaM.MemberID = opBOR.MemberID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
    ON anaAU.AuthorID = opAU.AuthorID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
    ON anaCAL.FullDate = opBOR.BorrowDate
--Exclusion of existing data
EXCEPT
SELECT
    anaBOR.BorrowID,
    anaBOR.BookKey,
    anaBOR.MemberKey,
    anaBOR.AuthorKey,
    anaBOR.CalendarKey,
    anaBOR.BorrowDuration,
    anaBOR.ReturnedLate
FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BORROWFACT anaBOR
```

INSERT INTO ReservationFact

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.RESERVATIONFACT
--SELECTION
SELECT
    opRES.ReservationID,
    anaB.BookKey,
    anaM.Memberkey,
    anaAU.AuthorKey,
    anaCAL.CalendarKey
FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Reservation opRES
--Grab reference to author
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
    ON opB.BookID = opRES.BookID
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
    ON opAU.AuthorID = opB.AuthorID
--Retrieve key values from warehouse
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
    ON anaB.BookID = opB.BookID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
    ON anaM.MemberID = opRES.MemberID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
    ON anaAU.AuthorID = opAU.AuthorID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
    ON anaCAL.FullDate = opRES.ReservationDate
--Exclusion of existing data
EXCEPT
SELECT
    anaRES.ReservationID,
    anaRES.BookKey,
    anaRES.MemberKey,
    anaRES.AuthorKey,
    anaRES.CalendarKey
FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.RESERVATIONFACT anaRES
```

INSERT INTO ReviewFact

```
INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.REVIEWFACT
--SELECTION
SELECT
    opREV.ReviewID,
    anaB.BookKey,
    anaM.Memberkey,
    anaAU.AuthorKey,
    anaCAL.CalendarKey,
    opREV.Rating,
    opREV.Comment
FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Review opREV
--Grab reference to author
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
    ON opB.BookID = opREV.BookID
JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
    ON opAU.AuthorID = opB.AuthorID
--Retrieve key values from warehouse
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
    ON anaB.BookID = opB.BookID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
    ON anaM.MemberID = opREV.MemberID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
    ON anaAU.AuthorID = opAU.AuthorID
JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
    ON anaCAL.FullDate = opREV.ReviewDate
--Exclusion of existing data
EXCEPT
SELECT
    anaREV.ReviewID,
    anaREV.BookKey,
    anaREV.MemberKey,
    anaREV.AuthorKey,
    anaREV.CalendarKey,
    anaREV.Rating,
    anaREV.Comment
FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.REVIEWFACT anaREV
```

SQL Code for ETL Procedures

The use of SQL procedures were used to ensure a clean and organized ETL infrastructure. When inserting data into the warehouse, we want to ensure that the tables without foreign key dependencies are the first to be populated, and then the dependency tables may be populated.

This may be done manually, but risks an improper insertion of data trying to reference records that do not yet exist. To avoid this risk, procedures for the ETL process were generated.

AI tools were used to brainstorm solutions to the ETL insertion order. Before consulting AI tools, we assumed that SQL functions would suffice in providing a structured order. Unfortunately, we learned that functions cannot perform insert operations, which is heavily required in the ETL process. The workaround is to use procedures, storing each part of the ETL insertion process in a separate procedure. From this, we designed a method of executing, or calling, each procedure in a *main* procedure, much like programming in Python or Java, which would control the order of table population. This system solved our issue of an improper insertion order, and we gained new experience in powerful SQL tools.

Create Procedure RefreshDimAuthor

```
CREATE PROCEDURE RefreshDimAuthor
AS
BEGIN
    --Insertion Query
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR (AuthorID, Name, Nationality, BirthYear)
        SELECT opA.AuthorID, opA.name, opA.Nationality, opA.BirthYear
            FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opA
            EXCEPT
            SELECT anaA.authorID, anaA.name, anaA.nationality, anaA.BirthYear
            FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaA
END;
```

Create Procedure RefreshDimBook

```
CREATE PROCEDURE RefreshDimBook
AS
BEGIN
    --Insertion into warehouse
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK
        --Selection of proper columns
        SELECT opB.BookID, opb.Title, opb.Genre, opb.PublishedYear, opb.ISBN
            FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
        --Exclusion of existing warehouse data
        EXCEPT
        SELECT anaB.BookID, anaB.Title, anaB.Genre, anaB.PublishedYear, anaB.ISBN
            FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
END;
```

Create Procedure RefreshDimCategory

```
CREATE PROCEDURE RefreshDimCategory
AS
BEGIN
    --Insertion into Category Dimension
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY
        --Row Selection
        SELECT opC.CategoryID, opC.CategoryName
        FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Category opC
        --Exclusion of existing rows
        EXCEPT
        SELECT anaC.CategoryID, anaC.CategoryName
        FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY anaC
END;
```

Create Procedure RefreshDimMember

```
CREATE PROCEDURE RefreshDimMember
AS
BEGIN
    --Insertion into Member dimension
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER
        --Row Selection
        SELECT opM.MemberID, opM.FirstName, opM.LastName, opM.Email, opM.PhoneNumber, opM.MembershipDate
        FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Member opM
        --Exclusion of existing data
        EXCEPT
        SELECT anaM.MemberID, anaM.FirstName, anaM.LastName, anaM.Email, anaM.PhoneNumber, anaM.MembershipDate
        FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
END;
```

Create Procedure RefreshFactBridgeBookCategory

```
CREATE PROCEDURE RefreshFactBridgeBookCategory
AS
BEGIN
    --Insertion into BRIDGEBOOKCATEGORYFACT
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BRIDGEBOOKCATEGORYFACT
        --Row Selection
        SELECT anaBook.BookKey, anaCategory.CategoryKey
            FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.BookCategory opBookCategory
            JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaBook
            ON anaBook.BookID = opBookCategory.BookID
            JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CATEGORY anaCategory
            ON anaCategory.CategoryID = opBookCategory.CategoryID
        --Exclusion of existing rows
        EXCEPT
        SELECT anaBBCF.BookKey, anaBBCF.CategoryKey
            FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BRIDGEBOOKCATEGORYFACT anaBBCF
END;
```

Create Procedure RefreshFactBorrow

```
CREATE PROCEDURE RefreshFactBorrow
AS
BEGIN
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BORROWFACT
        --SELECTION
        SELECT
            opBOR.BorrowID,
            anaB.BookKey,
            anaM.Memberkey,
            anaAU.AuthorKey,
            anaCAL.CalendarKey,
            DATEDIFF(DAY, opBOR.BorrowDate, opBOR.ReturnDate) AS BorrowDuration,
            CAST(CASE WHEN DATEDIFF(DAY, opBOR.BorrowDate, opBOR.ReturnDate) > 14 THEN 1 ELSE 0 END AS bit) AS ReturnedLate
        FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Borrowing opBOR
        --Grab reference to author
        JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
            ON opB.BookID = opBOR.BookID
        JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
            ON opAU.AuthorID = opB.AuthorID
        --Retrieve key values from warehouse
        JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
            ON anaB.BookID = opB.BookID
        JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
            ON anaM.MemberID = opBOR.MemberID
        JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
            ON anaAU.AuthorID = opAU.AuthorID
        JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
            ON anaCAL.FullDate = opBOR.BorrowDate
        --Exclusion of existing data
        EXCEPT
        SELECT
            anaBOR.BorrowID,
            anaBOR.BookKey,
            anaBOR.MemberKey,
            anaBOR.AuthorKey,
            anaBOR.CalendarKey,
            anaBOR.BorrowDuration,
            anaBOR.ReturnedLate
        FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BORROWFACT anaBOR
END;
```

Create Procedure RefreshFactReservation

```
CREATE PROCEDURE RefreshFactReservation
AS
BEGIN
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.RESERVATIONFACT
        --SELECTION
    SELECT
        opRES.ReservationID,
        anaB.BookKey,
        anaM.Memberkey,
        anaAU.AuthorKey,
        anaCAL.CalendarKey
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Reservation opRES
        --Grab reference to author
    JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
        ON opB.BookID = opRES.BookID
    JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
        ON opAU.AuthorID = opB.AuthorID
        --Retrieve key values from warehouse
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
        ON anaB.BookID = opB.BookID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
        ON anaM.MemberID = opRES.MemberID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
        ON anaAU.AuthorID = opAU.AuthorID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
        ON anaCAL.FullDate = opRES.ReservationDate
        --Exclusion of existing data
    EXCEPT
    SELECT
        anaRES.ReservationID,
        anaRES.BookKey,
        anaRES.MemberKey,
        anaRES.AuthorKey,
        anaRES.CalendarKey
    FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.RESERVATIONFACT anaRES
END;
```

Create Procedure RefreshFactReview

```
CREATE PROCEDURE RefreshFactReview
AS
BEGIN
    INSERT INTO [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.REVIEWFACT
        --SELECTION
    SELECT
        opREV.ReviewID,
        anaB.BookKey,
        anaM.Memberkey,
        anaAU.AuthorKey,
        anaCAL.CalendarKey,
        opREV.Rating,
        CAST(opREV.Comment AS NVARCHAR(MAX)) AS Comment
    FROM [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Review opREV
        --Grab reference to author
    JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Book opB
        ON opB.BookID = opREV.BookID
    JOIN [CITA 345 - Project 2 - Operational DB Library Management System].dbo.Author opAU
        ON opAU.AuthorID = opB.AuthorID
        --Retrieve key values from warehouse
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.BOOK anaB
        ON anaB.BookID = opB.BookID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.MEMBER anaM
        ON anaM.MemberID = opREV.MemberID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.AUTHOR anaAU
        ON anaAU.AuthorID = opAU.AuthorID
    JOIN [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.CALENDAR anaCAL
        ON anaCAL.FullDate = opREV.ReviewDate
        --Exclusion of existing data
    EXCEPT
    SELECT
        anaREV.ReviewID,
        anaREV.BookKey,
        anaREV.MemberKey,
        anaREV.AuthorKey,
        anaREV.CalendarKey,
        anaREV.Rating,
        CAST(anaREV.Comment AS NVARCHAR(MAX)) AS Comment
    FROM [CITA 345 - Project 2 - Analytical DB Library Management System].dbo.REVIEWFACT anaREV
END;
```

Create Procedure RefreshWarehouse

```
CREATE PROCEDURE RefreshWarehouse
AS
BEGIN
    --Refresh Dimensions without dependencies
    EXEC RefreshDimAuthor
    EXEC RefreshDimBook
    EXEC RefreshDimCategory
    EXEC RefreshDimMember

    --Refresh fact tables & tables with dependencies
    EXEC RefreshFactBridgeBookCategory
    EXEC RefreshFactBorrow
    EXEC RefreshFactReview
    EXEC RefreshFactReservation
END;
```

SQL Code for Analytics

The analytical queries below provide relevant information about how people are interacting within our library management system. The output of these queries can further be used in front-end business intelligence applications to generate visuals and statistics about the respective query. Each query will show its respective output. Something to notice about the outcomes is that the results appear to be uniform (see SQL Code for Randomized Entries). We concluded that this is an issue with our randomized SQL queries, and not an issue with our analytical queries.

AI tools were used to ensure the quality of specific SQL queries. Most notably, the query *Find the % of Late Returned Books per Month* posed challenges when designing due to the GROUP BY clause. Selected columns that are not part of the provided GROUP BY clause and not in an aggregate function are not allowed in SQL queries. This poses a problem for columns that we wanted to select but were not part of the grouping process. However, since grouping by constants has no logical effect on the output, and the columns creating problems were constants, we managed to group all selected columns and retrieve the correct output. The use of AI tools in this situation was to ensure that our logic was correct to ensure our output was correct. We learned

that while we could manually test the results, the process would take immense time, meanwhile AI can compute it within seconds.

Find the total number of books borrowed/month-year

	Year	Month	NumOfBooksBorrowed
1	2010	1	1185
2	2010	2	1116
3	2010	3	1225
4	2010	4	1190
5	2010	5	1264
6	2010	6	1132
7	2010	7	1240
8	2010	8	1208
9	2010	9	1186
10	2010	10	1189
11	2010	11	1186
12	2010	12	1178
13	2011	1	1182
14	2011	2	1140
15	2011	3	1216
16	2011	4	1123
17	2011	5	1227
18	2011	6	1116
19	2011	7	1149
20	2011	8	1197
21	2011	9	1213

Find the average borrowing duration per genre

	Genre	AvgBorrowDuration
1	Dystopian	37
2	Thriller	30
3	Non-Fiction	30
4	Romance	30
5	Horror	30
6	Fantasy	30
7	Historical	30
8	Science Fiction	30
9	Mystery	30

Find the % of late returned books per month

	Month	PercentageOfLateReturns
1	1	77
2	2	76
3	3	76
4	4	76
5	5	76
6	6	76
7	7	76
8	8	76
9	9	76
10	10	76
11	11	76
12	12	76

Find the average rating per genre

	Genre	AvgRating
1	Horror	2
2	Historical	2
3	Science Fiction	2
4	Mystery	2
5	Thriller	2
6	Romance	2
7	Fantasy	2
8	Dystopian	4
9	Non-Fiction	2

Identify Peak Borrowing Periods During the Week

	DayOfWeek	NumOfBooksBorrowed
1	Monday	43363

SQL Code for Randomized Entries

After testing the ETL infrastructure and analytical queries, we were ready to populate the tables with many random entries and perform a final test for the quality of our warehouse and queries. In total, we generated over 1,000,000 entries of data across all tables. Unfortunately, we noticed an issue with the randomization process while testing the analytical queries. While the process appears to be random, running analytical queries revealed that, on average, the output was very predictable. For the purposes of this project, we decided that the data provided would suffice. Further research into developing randomized entries in a database may be explored in future projects.

AI tools were used extensively in this section of our project, mainly due to the lack of experience in creating queries with randomized data. Our process for getting a proper query for our database required us to explain the table structure in our operational database and asking AI tools to generate INSERT queries based on the provided data types and value constraints (i.e., referential integrity constraints, integer between two given numbers, etc.). After some trial and error, we managed to receive insertion queries to populate our database with over 1,000,000 entries across all tables. Additionally, we learned a general structure we may follow to generate randomized data, which includes many programming techniques like variable declarations, while loops, and casting data from one type to another.

INSERT Many INTO Author

```

DECLARE @i INT = 4;

WHILE @i <= 100000
BEGIN
    INSERT INTO Author (AuthorID, Name, Nationality, BirthYear)
    VALUES (
        @i,
        -- Random name (First + Last)
        CONCAT(
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(varchar(36), NEWID()), 1, 6)),
            ' ',
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(varchar(36), NEWID()), 1, 6))
        ),
        -- Random nationality from a fixed list
        (SELECT TOP 1 val FROM
            (VALUES ('American'), ('British'), ('French'), ('Japanese'),
                ('German'), ('Canadian'), ('Brazilian'), ('Indian')) AS T(val)
            ORDER BY NEWID()
        ),
        -- Random birth year between 0000 and 2004
        ABS(CHECKSUM(NEWID())) % 2005 -- Random integer from 0 to 2004
    );

    SET @i = @i + 1;
END;

```

INSERT Many INTO Book

```
DECLARE @i INT = 4;

WHILE @i <= 200000
BEGIN
    INSERT INTO Book (BookID, Title, AuthorID, Genre, PublishedYear, ISBN)
    VALUES (
        @i,
        -- Random book title: "Book" + random string
        CONCAT('Book ',
            CHAR(65 + ABS(CHECKSUM(NEWID())) % 26),
            LOWER(SUBSTRING(CONVERT(varchar(36), NEWID()), 1, 5))
        ),
        -- Random AuthorID between 1 and 99999
        1 + ABS(CHECKSUM(NEWID())) % 99999,
        -- Random genre from list
        (SELECT TOP 1 val FROM
            (VALUES ('Fantasy'), ('Science Fiction'), ('Mystery'), ('Romance'),
                ('Horror'), ('Historical'), ('Non-Fiction'), ('Thriller')) AS G(val)
            ORDER BY NEWID()
        ),
        -- Random published year: 1450 to 2024
        1450 + ABS(CHECKSUM(NEWID())) % (2025 - 1450),
        -- Random 13-digit ISBN as BIGINT
        CAST(ABS(CHECKSUM(NEWID())) % 10000000000000 + 1000000000000 AS BIGINT)
    );

    SET @i = @i + 1;
END;
```

INSERT Many INTO Category

```
DECLARE @i INT = 3;

WHILE @i <= 100000
BEGIN
    INSERT INTO Category(CategoryID, CategoryName)
    VALUES (
        @i,
        -- Random name (First + Last)
        CONCAT(
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(varchar(36), NEWID()), 1, 6)),
            ' ',
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(varchar(36), NEWID()), 1, 6))
        )
    )
    SET @i = @i + 1;
END;
```

INSERT Many INTO Member

```
DECLARE @i INT = 3;

WHILE @i <= 300000
BEGIN
    INSERT INTO Member (MemberID, FirstName, LastName, Email, PhoneNumber, MembershipDate)
    VALUES (
        @i,
        -- Random first name
        CONCAT(
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(VARCHAR(36), NEWID()), 1, 5))
        ),
        -- Random last name
        CONCAT(
            CHAR(65 + ABS(CHECKSUM(NEWID()))) % 26),
            LOWER(SUBSTRING(CONVERT(VARCHAR(36), NEWID()), 1, 7))
        ),
        -- Random email
        CONCAT(
            LOWER(SUBSTRING(CONVERT(VARCHAR(36), NEWID()), 1, 8)),
            '@example.com'
        ),
        -- Random 10-digit phone number (U.S. style)
        CAST(1000000000 + ABS(CHECKSUM(NEWID())) % 9000000000 AS BIGINT),
        -- Random date between 2010-01-01 and 2030-12-31
        DATEADD(DAY, ABS(CHECKSUM(NEWID())) % DATEDIFF(DAY, '2010-01-01', '2030-12-31'), '2010-01-01')
    );

    SET @i = @i + 1;
END;
```

INSERT Many INTO BookCategory

```
DECLARE @i INT = 1;
DECLARE @BookID INT;
DECLARE @CategoryID INT;

WHILE @i <= 500000
BEGIN
    SET @BookID = 1 + ABS(CHECKSUM(NEWID())) % 200000;
    SET @CategoryID = 1 + ABS(CHECKSUM(NEWID())) % 100000;

    -- Only insert if the pair doesn't already exist
    IF NOT EXISTS (
        SELECT 1 FROM BookCategory
        WHERE BookID = @BookID AND CategoryID = @CategoryID
    )
    BEGIN
        INSERT INTO BookCategory(BookID, CategoryID)
        VALUES (@BookID, @CategoryID);

        SET @i = @i + 1;
    END
END;
```

INSERT Many INTO Borrowing

```
DECLARE @MaxBookID INT = 200000;
DECLARE @MaxMemberID INT = 300000;
DECLARE @i INT = 3;

WHILE @i <= 300000
BEGIN
    -- Common values
    DECLARE @BookID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxBookID;
    DECLARE @MemberID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxMemberID;
    DECLARE @BaseDate DATE = DATEADD(DAY, ABS(CHECKSUM(NEWID())) % DATEDIFF(DAY, '2010-01-01', '2030-12-31'), '2010-01-01');

    -- Insert Borrowing (ReturnDate after BorrowDate)
    DECLARE @ReturnDate DATE = DATEADD(DAY, 1 + ABS(CHECKSUM(NEWID())) % 60, @BaseDate); -- 1 to 60 days later

    INSERT INTO Borrowing (BorrowID, BookID, MemberID, BorrowDate, ReturnDate)
    VALUES (
        @i,
        @BookID,
        @MemberID,
        @BaseDate,
        @ReturnDate
    );

    SET @i = @i + 1;
END;
```

INSERT Many INTO Reservation

```
DECLARE @MaxBookID INT = 200000;
DECLARE @MaxMemberID INT = 300000;
DECLARE @i INT = 2;

WHILE @i <= 300000
BEGIN
    -- Common values
    DECLARE @BookID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxBookID;
    DECLARE @MemberID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxMemberID;
    DECLARE @BaseDate DATE = DATEADD(DAY, ABS(CHECKSUM(NEWID())) % DATEDIFF(DAY, '2010-01-01', '2030-12-31'), '2010-01-01');

    -- Insert Reservation
    INSERT INTO Reservation (ReservationID, BookID, MemberID, ReservationDate)
    VALUES (
        @i,
        @BookID,
        @MemberID,
        @BaseDate
    );

    SET @i = @i + 1;
END;
```

INSERT Many INTO Review

```
DECLARE @MaxBookID INT = 200000;
DECLARE @MaxMemberID INT = 300000;

DECLARE @i INT = 3;

WHILE @i <= 300000
BEGIN
    -- Common values
    DECLARE @BookID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxBookID;
    DECLARE @MemberID INT = 1 + ABS(CHECKSUM(NEWID())) % @MaxMemberID;
    DECLARE @BaseDate DATE = DATEADD(DAY, ABS(CHECKSUM(NEWID())) % DATEDIFF(DAY, '2010-01-01', '2030-12-31'), '2010-01-01');

    -- Insert Review
    INSERT INTO Review (ReviewID, BookID, MemberID, Rating, Comment, ReviewDate)
    VALUES (
        @i,
        @BookID,
        @MemberID,
        ABS(CHECKSUM(NEWID())) % 6, -- 0-5 rating
        CONCAT('This is a random review comment #', @i),
        @BaseDate
    );

    SET @i = @i + 1;
END;
```

Reflection

Working on the Library Management System project, particularly the Analytics and ETL components, has provided a deep and practical understanding of how data flows from operational databases into analytical environments to support decision-making. The ETL process was a critical aspect, involving careful data extraction, transformation, and loading to ensure consistency and accuracy in the analytical database. Through writing SQL statements that eliminate duplicates using EXCEPT clauses, and constructing a recursive calendar table, we gained valuable hands-on experience in data cleansing, historical data management, and dimension modeling.

The star schema and ERD design clarified the relationships between fact and dimension tables, which are essential for supporting advanced queries. By mapping operational data into fact tables like BORROWFACT, RESERVATIONFACT, and REVIEWFACT, we created a foundation for meaningful analytics. Creating bridge tables such as BRIDGEBOOKCATEGORYFACT further emphasized the importance of handling many-to-many relationships in a warehouse schema.

Running analytical queries demonstrated the power of the data warehouse model. For example, identifying trends like the average borrowing duration by genre or peak borrowing days of the week provided actionable insights that could improve library operations. One particularly interesting metric was the percentage of late returns per month, which required joining subqueries and performing calculations, this helped us appreciate the complexity and usefulness of data aggregation.

We gained many insights into the power of AI tools in ensuring project quality. Through AI, we managed to research star schema principles, ETL infrastructure solutions, and data randomization in a manner of minutes instead of days. More importantly, we learned new, powerful techniques for data warehousing that will carry over with us in the future. Additionally, we gained new skills in researching through AI that is unprecedented. This project provided a great activity in helping us learn the place of AI in a work-like environment.

Overall, this project deepened our technical proficiency with SQL, improved our understanding of data warehousing principles, and reinforced the importance of clean, well-modeled data for analytics. It also highlighted the real-world relevance of ETL pipelines in bridging operational systems with analytical needs. This hands-on experience has undoubtedly prepared us for more advanced work in data engineering and analytics.